차세대 웹 기반 3D 인테리어 디자인 플랫폼 구축을 위한 제로 투 원(Zero-to-One) 종합 기술 분석 및 실행 전략 보고서

1. 서론: 프로토타입과 상용 제품 간의 기술적 간극 분석

1.1. 현재 상황 진단 및 사용자의 불만 요인 분석

사용자가 제기한 "작업을 수행했으나 체감상 변화가 없고 UI가 미려하지 않다"는 문제는 현대 웹 3D 개발, 특히 React Three Fiber(R3F)와 WebGL을 활용한 프로젝트에서 매우 빈번하게 발생하는 현상이다. 사용자가 제공한 working.pdf와 대화 기록을 분석해본 결과, 현재 프로젝트는 기능적 구현(Functional Implementation) 단계에는 도달했으나 경험적 최적화(Experiential Optimization) 단계로 넘어가지 못한 전형적인 '튜토리얼의 함정(Tutorial Trap)'에 빠져 있는 것으로 판단된다.

일반적인 튜토리얼이나 기술 문서는 "3D 객체를 화면에 띄우는 법"이나 "데이터베이스에 좌표를 저장하는 법"과 같은 개별적인 기술 단위의 구현에 집중한다. 그러나 Dverso Studio나 Caffe.design과 같이 사용자가 레퍼런스로 제시한 하이엔드 웹사이트들은 이러한 개별 기술들이 고도로 정제된 렌더링 파이프라인(Rendering Pipeline), 포스트 프로세싱(Post-processing), 그리고 사용자 인터랙션(UX) 설계와 결합되어 유기적으로 작동할 때 비로소 완성된다. 현재 사용자가 느끼는 "구리다"는 감상은 코드의 오류라기보다는, 상용 수준의 시각적 품질을 결정짓는 조명, 질감, 카메라 움직임의 미세 조정(Polishing)이 부재하기 때문에 발생하는 필연적인 결과다.

특히 사용자가 가장 중요하게 언급한 **'도면 인식 정확도 및 3D 맵 생성'**의 실패는 단순한 코딩의 문제가 아니라, 비정형 데이터를 정형 데이터로 변환하는 알고리즘적 접근 방식의 한계에서 기인한다. 기존에 시도했던 Gemini Flash 모델이나 단순 이미지 분석은 건축 도면이 가진 복잡한 노이즈(치수 선, 가구 기호 등)를 효과적으로 필터링하지 못했을 가능성이 크다. 이를 해결하기 위해서는 단순한 이미지 인식을 넘어선 **의미론적 세그멘테이션(Semantic Segmentation)**과 **기하학적 보정(Geometric Correction)**이 결합된 하이브리드 파이프라인이 필수적이다.

1.2. 전략적 방향성: 리팩토링(Refactoring) 대 재작성(Rewrite)

사용자는 현재 작업 중인 프로젝트 위에서 계속 진행할지, 아니면 새로운 디렉토리에서 시작할지를 고민하고 있다. 제공된 파일 구조 정보([1])를 면밀히 분석한 결과, 현재 프로젝트는 apps/web, packages/shared, docs 등으로 구성된 모노레포(Monorepo) 구조를 취하고 있다. 이는 대규모 애플리케이션 확장성과 유지보수성을 고려한 매우 현대적이고 견고한 아키텍처다. Supabase와의 데이터베이스 연동, 기본적인 인증 로직, 그리고 R3F의 기본 설정이 이미 되어 있는 상태에서 이를 버리고 새로 시작하는 것은 막대한 시간 낭비를 초래할 뿐만 아니라, 동일한

초기 설정 단계를 반복하게 만듦으로써 프로젝트의 모멘텀을 저하시킬 위험이 있다.

따라서 본 보고서는 기존 프로젝트의 구조를 유지하되, 핵심 엔진을 교체하는 '소프트 리셋(**Soft Reset**)' 전략을 강력히 권고한다. 기존의 디렉토리 구조와 인프라(Auth, DB 연결, 라우팅)는 그대로 활용하되, 사용자가 불만족스러워하는 시각적 요소(Scene Graph)와 도면 분석 로직(Analysis Engine)은 과감하게 모듈 단위로 재설계하여 대체하는 방식이다. 이는 "Zero to One"을 달성하기 위한 가장 효율적이고 실용적인 접근법이다.

---

2. 하이엔드 3D 웹 애플리케이션을 위한 기술 아키텍처 설계

사용자가 목표로 하는 Dverso Studio 수준의 퀄리티를 달성하기 위해서는 웹사이트가 아닌 **'웹 기반 공간 운영체제(Spatial OS)'**를 만든다는 관점으로 접근해야 한다. 이를 위해 2025년 기준, 가장 최적화되고 트렌디한 기술 스택과 그 선정 이유를 상세히 기술한다.

2.1. 핵심 기술 스택 및 선정 근거

계층 (**Layer**)	기술 스택 (**Technology**)	선정 근거 및 최적화 포인트
프레임워크	**Next.js 14+ (App Router)**	단순한 SPA가 아닌, SEO와 초기 로딩 속도(FCP)를 최적화하기 위해 필수적이다. 특히 AI 분석과 같은 무거운 연산을 서버리스 함수(Serverless Functions)로 처리하여 클라이언트의 부하를 줄이고 보안을 강화할 수 있다.
3D 엔진	**React Three Fiber (R3F)**	Three.js를 React 생태계에 통합한 라이브러리로, 복잡한 3D 씬(Scene)을 컴포넌트 단위로 관리할 수 있게 해준다. 이는 나중에 설명할 '가구 배치'나 '벽 생성'과 같은 동적 기능을 선언적으로 관리하는 데 있어 바닐라 Three.js보다

		월등한 생산성을 제공한다.[2]
상태 관리	**Zustand**	매우 중요. Redux나 Context API는 3D 렌더링 루프(60fps)와 결합될 때 불필요한 리렌더링을 유발하여 치명적인 성능 저하(렉)를 일으킨다. Zustand는 React 컴포넌트 외부에서 상태를 관리하고 필요할 때만 컴포넌트를 구독시키는 과도적 업데이트(Transient Updates)가 가능하여, FPS 게임과 같은 부드러운 움직임을 구현하는 데 필수적이다.[1]
물리 엔진	**Rapier.js (@react-three/rapier)**	사용자가 요청한 '1인칭 시점에서의 벽 충돌 방지'를 구현하기 위해 필수적이다. 기존의 Cannon.js보다 월등히 빠른 WebAssembly(WASM) 기반의 물리 엔진으로, 웹 브라우저에서도 정교한 충돌 감지와 물리 상호작용을 끊김 없이 처리할 수 있다.[4]
AI 비전	**Claude 3.5 Sonnet / GPT-4o**	도면 인식의 정확도를 높이기 위해 단순 OCR이나 OpenCV만으로는 부족하다. 이미지 내의 '방', '문', '창문'의 맥락을 이해하는 멀티모달 LLM을 활용하여 구조화된 JSON 데이터를 추출하는 방식을 채택한다.[5]
스타일링	**Tailwind CSS + Framer Motion**	Dverso Studio와 같은 '트렌디한' 느낌은 마이크로 인터랙션에서 온다. Framer

		Motion은 3D 캔버스 위에 떠 있는 UI(HUD)의 진입/진출 애니메이션을 물리 기반으로 자연스럽게 처리해주며, Tailwind는 빠른 디자인 시스템 구축을 돕는다.[6]

2.2. 확장성을 고려한 디렉토리 구조 재설계

기존 프로젝트 위에서 작업을 진행하되, 코드의 복잡도를 낮추고 유지보수성을 높이기 위해 다음과 같은 도메인 주도(Domain-Driven) 디렉토리 구조로의 리팩토링을 제안한다. 이는 사용자가 겪은 "어디를 고쳐야 할지 모르겠다"는 문제를 해결해 줄 것이다.

Plaintext

```
apps/web/src/
├── app/              # Next.js App Router (페이지 라우팅)
│   ├── (auth)/       # 로그인, 회원가입 등 인증 관련 페이지
│   ├── (dashboard)/  # 프로젝트 목록, 대시보드
│   └── editor/[id]/  # 핵심 3D 에디터 페이지
├── components/
│   ├── canvas/       # R3F 관련 3D 컴포넌트 (Scene Graph)
│   │   ├── effects/  # Bloom, Noise, SSAO 등 포스트 프로세싱 효과
│   │   ├── controls/ # 1인칭(FPS), 탑뷰(Orbit) 컨트롤러 로직
│   │   ├── environment/ # HDRI 조명, 그림자 설정
│   │   └── features/ # Wall, Floor, FurnitureMesh 등 실제 객체
│   ├── overlay/      # 3D 위에 띄워질 2D UI (인벤토리, 속성 패널)
│   └── shared/       # 버튼, 모달 등 공용 UI
├── hooks/            # useKeyboard, useAssetLoader 등 커스텀 훅
├── lib/
│   ├── ai/           # 도면 분석을 위한 AI API 호출 로직
│   ├── geometry/     # 2D 좌표를 3D 메쉬로 변환하는 수학적 알고리즘
│   └── stores/       # Zustand 스토어 (editorStore, playerStore)
└── services/         # Supabase 데이터베이스 통신 로직
```

---

3. 핵심 엔진 구현: 도면 인식의 정확도 향상 및 3D 맵 생성

사용자가 가장 큰 어려움을 겪고 있는 '도면 분석 후 **3D** 모델링' 과정은 이 프로젝트의 기술적 해자(Moat)가 될 수 있는 핵심 기능이다. 기존의 방식이 실패한 이유는 도면 이미지를 단순히 픽셀 덩어리로 처리했거나, AI에게 모호한 지시를 내렸기 때문일 가능성이 높다. 이를 해결하기 위해 **'의미론적 파싱(Semantic Parsing) 후 기하학적 재구성(Geometric Reconstruction)'**이라는 2단계 접근법을 제시한다.

3.1. 1단계: AI를 활용한 의미론적 구조 추출 (Semantic Extraction)

도면 이미지를 3D로 바로 변환하려고 하면 AI는 환각(Hallucination)을 일으켜 비현실적인 모델을 생성한다. 대신, AI에게는 이미지를 해석하여 구조화된 데이터**(JSON)**로 변환하는 임무만을 부여해야 한다.

- 프롬프트 엔지니어링 전략:
  도면 이미지를 Claude 3.5 Sonnet API(또는 GPT-4o)에 전송할 때, 다음과 같은 시스템 프롬프트를 사용하여 기하학적 데이터를 요청해야 한다.

  > "이 평면도 이미지를 분석하여 건축학적 위상(topology)을 추출하라.
  > 결과는 다음 JSON 스키마를 엄격히 따라야 한다:
  > 1. walls: 각 벽의 시작점과 끝점 좌표 [x1, y1, x2, y2]의 배열. (이미지 좌상단을 0,0 우하단을 100,100으로 정규화할 것)
  > 2. openings: 문과 창문의 중심 좌표, 너비, 그리고 유형(door 또는 window).
  > 3. scale: 표준 문의 너비(900mm)를 기준으로 픽셀 대 미터 비율을 추정하라."

이 방식은 AI가 시각적 노이즈(가구 그림, 텍스트)를 무시하고 벽의 선(Line)이라는 핵심 정보만 추출하게 만든다.[5]

3.2. 2단계: 휴먼 인 더 루프(Human-in-the-Loop) 보정 인터페이스

가장 중요한 통찰**:** 현재 기술로 AI가 도면을 100% 완벽하게 좌표로 변환하는 것은 불가능하다. 사용자가 "정확도가 안 나온다"고 느끼는 것은 AI의 결과물을 수정할 기회 없이 바로 3D로 렌더링했기 때문이다. 따라서 **보정 단계(Verification Step)**가 반드시 필요하다.

- **2D** 에디터 구현**:** AI가 반환한 JSON 데이터를 바탕으로, 원본 도면 이미지 위에 SVG나 HTML5 Canvas로 선을 그려주는 2D 인터페이스를 먼저 보여줘야 한다.
- 기능**:** 사용자가 AI가 잘못 인식한 벽의 끝점을 드래그하여 코너에 정확히 맞추거나(Snap), 누락된 벽을 추가할 수 있게 한다. 이 과정을 거쳐야만 비로소 '건축적으로 올바른' 3D 모델을 생성할 수 있다.[8]

3.3. 3단계: 절차적 3D 메쉬 생성 (Procedural Mesh Generation)

보정된 2D 벡터 데이터를 기반으로 Three.js에서 3D 벽을 생성한다.

- 벽 생성 **(Extrusion):** THREE.Shape로 2D 경로를 정의하고 THREE.ExtrudeGeometry를 사용하여 높이(예: 2400mm)만큼 돌출시킨다.
- 창문**/**문 뚫기 **(CSG):** 벽 메쉬 위에 문 모델을 단순히 겹쳐 놓으면 안 된다. three-bvh-csg와 같은 라이브러리를 사용하여 벽 메쉬에서 문/창문의 부피만큼을 연산적으로

빼내는(Subtract) **CSG(Constructive Solid Geometry)** 작업을 수행해야 한다. 그래야 1인칭 시점에서 창문 밖을 내다볼 수 있고, 문을 통해 방을 이동할 수 있다.[9]

---

4. 시각적 완성도: '트렌디'한 룩앤필(Look & Feel) 구현 전략

사용자가 언급한 Dverso Studio나 Caffe.design의 비주얼은 단순한 모델링이 아니라 **조명(Lighting)**과 **포스트 프로세싱(Post-processing)**의 조화에서 나온다. "UI가 구리다"는 평가는 대부분 밋밋한 조명과 그림자, 그리고 날것의 텍스처 때문이다.

4.1. 물리 기반 렌더링(PBR) 조명 시스템 구축

기본적인 AmbientLight는 공간을 평면적으로 보이게 만든다. 깊이감 있는 공간을 위해 다음 요소를 적용해야 한다.

1. 환경 맵 **(HDRI):** @react-three/drei의 Environment 컴포넌트를 사용하여 실제 공간의 빛 정보를 담은 HDRI(High Dynamic Range Image)를 적용한다. 이는 가구의 금속 재질이나 바닥의 반사광을 리얼하게 만들어준다. 'City', 'Apartment', 'Studio' 등의 프리셋을 상황에 맞춰 변경할 수 있도록 한다.[10]
2. 소프트 쉐도우 **(Soft Shadows):** 현실의 그림자는 칼처럼 날카롭지 않다. ContactShadows나 AccumulativeShadows를 사용하여 바닥에 닿는 부분은 진하고 멀어질수록 흐려지는 부드러운 그림자를 생성해야 한다. 이는 물체가 둥둥 떠 있는 느낌을 없애준다.[12]
3. 앰비언트 오클루전 **(AO):** 벽과 바닥이 만나는 모서리나 가구 틈새에 그림자를 더해주는 SSAO(Screen Space Ambient Occlusion) 효과를 적용하여 공간의 입체감을 극대화한다.[13]

4.2. 영화 같은 포스트 프로세싱 파이프라인

사용자가 원하는 '트렌디함'의 핵심은 바로 필터 효과다. @react-three/postprocessing 라이브러리를 사용하여 다음 효과 체인을 구성한다.[2]

1. **Bloom (빛 번짐):** 창문 밖에서 들어오는 빛이나 조명 기구에 Bloom 효과를 주어 몽환적이고 밝은 분위기를 연출한다. 단, 과도하지 않게 임계값(Threshold)을 조절하여 하이라이트 부분만 은은하게 빛나도록 한다.[15]
2. **Noise (필름 그레인):** 아주 미세한 노이즈(Noise) 효과를 전체 화면에 깐다. 이는 디지털 3D 특유의 인위적인 매끄러움을 상쇄시키고, 잡지 사진과 같은 질감을 부여한다.[17]
3. **Vignette (비네팅):** 화면 가장자리를 어둡게 하여 사용자의 시선을 화면 중앙의 콘텐츠로 집중시킨다.
4. **Chromatic Aberration (색수차):** 렌즈의 가장자리에서 색이 살짝 어긋나는 효과를 주어, 실제 카메라로 촬영한 듯한 리얼리티를 부여한다. Dverso Studio와 같은 사이트에서 자주 사용하는 기법이다.

4.3. UI 디자인 시스템: Glassmorphism & Motion

참고 이미지의 UI는 3D 화면을 가리지 않으면서 정보가 떠 있는 형태다.

- 스타일링: Tailwind CSS를 사용하여 bg-black/10 backdrop-blur-md border border-white/20와 같은 클래스로 반투명한 유리 질감의 패널을 만든다. 이는 배경의 3D 공간이 UI 뒤로 비치게 하여 공간감을 해치지 않는다.
- 애니메이션: 로딩 화면에서 대시보드로 넘어갈 때, Framer Motion의 AnimatePresence를 사용하여 UI 요소들이 순차적으로 부드럽게 등장(Stagger Children)하도록 한다. 이는 단순한 화면 전환이 아닌, 하나의 시퀀스처럼 느껴지게 한다.[6]

---

5. 인터랙션 메카니즘: 에디터 모드와 워크 모드의 이원화

사용자는 가구를 배치하는 CAD 스타일의 '탑뷰(Top-view)'와 직접 돌아다니는 '1인칭 시점(POV)'을 모두 원하고 있다. 이 두 가지 경험은 조작 방식과 카메라 로직이 완전히 다르므로, 하나의 상태(State)로 관리하여 모드를 전환하는 구조가 필요하다.

5.1. 모드 전환 아키텍처 (State Machine)

Zustand 스토어에 viewMode 상태('edit' | 'walk')를 정의하고, 이에 따라 렌더링되는 컨트롤러 컴포넌트를 교체한다.

- 편집 모드 **(Edit/Top-down):**
  - 카메라: OrthographicCamera (원근감이 없는 직교 투영)를 사용하여 가구의 크기 비례를 정확하게 파악하고 정렬하기 쉽게 한다.
  - 컨트롤: MapControls (마우스 우클릭으로 팬, 휠로 줌)를 사용한다.
  - 천장 처리: 이 모드에서는 천장(Ceiling) 메쉬의 visible 속성을 false로 설정하여 내부를 볼 수 있게 한다.
  - 상호작용: TransformControls를 사용하여 가구를 클릭했을 때 이동(Translate), 회전(Rotate) 핸들이 나타나도록 한다. 드래그 앤 드롭 시 레이캐스팅(Raycasting)을 통해 마우스 커서가 바닥 평면과 교차하는 지점을 계산하여 가구를 배치한다.[18]

5.2. 보행 모드 (Walk/FPS)

- 카메라: PerspectiveCamera (원근감 있는 투영)를 사용하며, 눈높이(y=1.6m)에 위치시킨다.
- 컨트롤: PointerLockControls를 사용하여 마우스 커서를 숨기고 시점을 회전시킨다.
- 물리 및 충돌: Rapier 물리 엔진의 RigidBody를 플레이어(캡슐 형태)와 벽(정적 형태)에 적용한다. 단순히 카메라 위치를 옮기는 것이 아니라, WASD 키 입력에 따라 플레이어의 RigidBody에 **속도(Velocity)**를 가하는 방식으로 구현해야 한다. 그래야 벽에 부딪혔을 때 뚫고 지나가지 않고 미끄러지는 자연스러운 움직임이 가능하다.[19]
- 상호작용 표시: 화면 중앙에 보이지 않는 레이캐스터(Raycaster)를 쏘아, 문이나 조명 스위치 등 상호작용 가능한 객체에 닿았을 때 크로스헤어(Crosshair)가 변경되거나 "E키를 눌러 상호작용"과 같은 툴팁이 뜨도록 구현한다.[1]

---

6. 단계별 실행 가이드 (Implementation Roadmap)

현재의 정체된 상황을 타개하고 완성형 서비스로 나아가기 위한 구체적인 실행 순서이다.

1단계: 기반 다지기 및 에셋 준비 (Foundation)

1. 레거시 정리: 기존 Scene3D 관련 코드를 백업 후 초기화한다.
2. 패키지 설치: npm install three @types/three @react-three/fiber @react-three/drei @react-three/postprocessing @react-three/rapier zustand framer-motion lucide-react
3. 에셋 확보: Poly Haven이나 Sketchfab에서 무료 GLB 가구 모델(의자, 테이블, 조명 등)과 텍스처(바닥, 벽지)를 다운로드하여 public/assets 폴더에 구조화하여 저장한다. GLB 파일은 gltf-pipeline 등을 통해 Draco 압축을 적용하여 로딩 속도를 최적화한다.[20]

2단계: 3D 뷰어 및 환경 구축 (The Stage)

1. **Canvas** 구성: R3F Canvas를 전체 화면으로 설정하고, shadows, dpr (디바이스 픽셀 비율) 설정을 추가한다.
2. 조명 및 환경: Environment와 ContactShadows를 배치하여 기본 씬의 때깔을 잡는다.
3. 포스트 프로세싱: EffectComposer 안에 Bloom, Noise, Vignette를 추가하고 값을 미세 조정하여 Dverso 느낌을 낸다.

3단계: 도면 분석 및 맵 생성 파이프라인 (The Core)

1. **API** 개발: 사용자가 업로드한 이미지를 받아 Claude 3.5 Sonnet API로 전송하고, 벽 좌표 JSON을 반환받는 Next.js API Route를 작성한다.
2. **2D** 보정 툴: React와 HTML5 Canvas API를 사용하여 이미지 위에 선을 그리고 수정할 수 있는 간단한 에디터를 만든다.
3. **3D** 생성기: 보정된 데이터를 받아 ExtrudeGeometry와 CSG 라이브러리를 통해 3D 벽과 바닥을 동적으로 생성하는 컴포넌트를 구현한다.

4단계: 인터랙션 및 게임플레이 구현 (The Experience)

1. 스토어 설정: Zustand를 사용하여 mode (edit/walk), furnitureList, playerPos 등의 전역 상태를 관리한다.
2. 컨트롤러 구현: EditControls와 WalkControls 컴포넌트를 각각 만들고 mode 상태에 따라 조건부 렌더링한다.
3. 가구 배치: 탑뷰 모드에서 드래그 앤 드롭으로 가구를 배치하고, 해당 가구의 좌표를 상태에 업데이트하는 로직을 완성한다.

5단계: UI 폴리싱 및 배포 (The Polish)

1. 로딩 화면: Suspense와 useProgress (Drei)를 사용하여 3D 에셋이 로딩되는 동안 트렌디한 프로그레스 바나 로고 애니메이션을 보여준다.
2. 랜딩 페이지: 3D 씬을 배경으로 깔고(투명도 조절), 그 위에 HTML/CSS로 타이포그래피 중심의 랜딩 페이지를 구성한다.

3. 최적화: React.memo를 사용하여 불필요한 리렌더링을 막고, 텍스처 해상도를 조절하여 프레임 드랍을 방지한다.

---

7. 결론 및 제언

사용자가 현재 느끼는 불만족은 개발 역량의 부족이 아니라, **상용 서비스 수준의 디테일(Polishing)**과 복합적인 기술 스택의 오케스트레이션이 부재했기 때문이다. 특히 도면 인식의 정확도 문제는 AI의 확률적 추론에만 의존해서는 해결할 수 없으며, 반드시 사용자의 검증 과정을 UI에 포함시켜야 한다.

본 보고서에서 제시한 대로 기존 모노레포 구조를 활용하여 백엔드(Auth, DB)의 안정성을 유지하면서, 프론트엔드 3D 엔진과 데이터 처리 파이프라인을 **물리 기반 렌더링(PBR)**과 하이브리드 **AI** 접근법으로 고도화한다면, Dverso Studio나 Caffe.design과 같은 수준 높은 시각적 경험과 기능적 완성도를 갖춘 플랫폼을 충분히 구축할 수 있을 것이다. 지금은 코드를 처음부터 다시 짜는 것이 아니라, 렌더링의 질을 높이고 데이터 흐름을 정교하게 다듬는 '질적 전환'이 필요한 시점이다.

Works cited

1. working.pdf
2. Post processing - Wawa Sensei, accessed December 29, 2025, https://wawasensei.dev/courses/react-three-fiber/lessons/post-processing
3. Learn the Basics of React Three Fiber by Building a Three-Point Lighting Setup, accessed December 29, 2025, https://codeworkshop.dev/blog/2019-12-31-learn-the-basics-of-react-three-fiber-by-building-a-three-point-lighting-setup
4. Bruno Simon, accessed December 29, 2025, https://bruno-simon.com/
5. How can I detect walls, doors, and windows to extract room data from complex floor plans? : r/learnmachinelearning - Reddit, accessed December 29, 2025, https://www.reddit.com/r/learnmachinelearning/comments/1o7less/how_can_i_detect_walls_doors_and_windows_to/
6. dverso studio - Web Design & Development, accessed December 29, 2025, https://dversostudio.io/
7. FloorPlan to SVG Converter - AnotherWrapper, accessed December 29, 2025, https://anotherwrapper.com/tools/ai-app-generator/floorplan-to-svg-converter
8. Opencv: Convert floorplan image into data model - Stack Overflow, accessed December 29, 2025, https://stackoverflow.com/questions/20429052/opencv-convert-floorplan-image-into-data-model
9. akramguediri/FloorPlan3DConverter: convert a floor plan to an actual 3d rendering - GitHub, accessed December 29, 2025, https://github.com/akramguediri/FloorPlan3DConverter
10. React-Three-Fiber: Enhancing Scene Quality with Drei + Performance Tips -

Medium, accessed December 29, 2025, https://medium.com/@ertugrulyaman99/react-three-fiber-enhancing-scene-quality-with-drei-performance-tips-976ba3fba67a
11. How to improve your 3D lighting with HDRIs in react three fiber - YouTube, accessed December 29, 2025, https://www.youtube.com/watch?v=nR7o0R-qJc4
12. project to enhance your scene's baked lighting - demo page & GitHub in comments : r/threejs - Reddit, accessed December 29, 2025, https://www.reddit.com/r/threejs/comments/126tp9n/project_to_enhance_your_scenes_baked_lighting/
13. Bloom - React Postprocessing, accessed December 29, 2025, https://react-postprocessing.docs.pmnd.rs/effects/bloom
14. postprocessing for react-three-fiber - GitHub, accessed December 29, 2025, https://github.com/pmndrs/react-postprocessing
15. Bloom - React Three Fiber Tutorials, accessed December 29, 2025, https://sbcode.net/react-three-fiber/bloom/
16. Three Fibre: BLOOM - How to set different intensity for different objects? : r/threejs - Reddit, accessed December 29, 2025, https://www.reddit.com/r/threejs/comments/1ei5q98/three_fibre_bloom_how_to_set_different_intensity/
17. Adding film grain ? : r/threejs - Reddit, accessed December 29, 2025, https://www.reddit.com/r/threejs/comments/11kg1j5/adding_film_grain/
18. 13 Best Floor Plan Software for Architects and Designers (2025) | ArchiVinci, accessed December 29, 2025, https://www.archivinci.com/blogs/best-floor-plan-softwares
19. How I made my 3d Portfolio Website | by Matt Park - Medium, accessed December 29, 2025, https://medium.com/@mattparkarchive/how-i-made-my-3d-portfolio-website-50b413a75541
20. Baked lighting in r3f - @tchayen, accessed December 29, 2025, https://tchayen.github.io/posts/baked-lighting-in-r3f